

a) Where can I find a copy of your video? If in uLearn, just say so

- ULearn

b) Give a description of the originally proposed functionality for your project, as well as details on what changed (things you did not complete or things you added). What caused the change? This is a good place to include user stories, sequence diagrams, timing diagrams, user flow diagrams, flowcharts of algorithms, etc. All documentation should be embedded inside the Word doc as an image.

- The originally proposed functionality of the project included watering reminders for users' plants, adding and deleting plants from a personal garden, searching for plants to add, and user authentication through login and registration. One major change involved how the app handles environmental data. Initially, the goal was to incorporate detailed weather-based factors such as UV index, temperature, and soil moisture to dynamically adjust plant care. However, this was scaled back to a simpler implementation where the app detects rain and automatically waters plants. This change was necessary due to limitations in available API data, inconsistencies across sources, and restrictions on API call limits, which made the original approach impractical to implement reliably.

c) In what areas did things not go well? What parts went exceptionally well?

- One challenge encountered was learning and properly implementing Android architecture components such as fragments, ViewModels, and repositories. This initially slowed development, but once a consistent structure was established, progress became much smoother. The backend development using PHP went particularly well, especially in handling API communication and database interactions. A significant issue, however, was the quality and limitations of external plant data APIs. Some APIs lacked complete or consistent data, and rate limits restricted usage, which ultimately required integrating multiple APIs to achieve the desired functionality.

d) What technology stack did you use for your project? Why did you choose this option? Give supporting details with specific reasons, whether human or technology.

- The project uses a Kotlin-based Android frontend, a PHP backend, an SQL database, and external APIs. Kotlin was chosen to gain experience with a modern language used in Android development, while still leveraging prior knowledge of Java. PHP and SQL were selected due to prior experience and their reliability for handling server-side logic and structured data storage. External APIs were chosen based on the scope of plant data they provided, although limitations in their data and usage constraints influenced some design decisions during development.

e) Does your project include at least two platforms? Identify them and the communication that happens between them.

- The project consists of three platforms: a Kotlin Android frontend, a PHP backend, and an SQL database. The Android application communicates with the backend via HTTP requests to retrieve and send data. The PHP backend processes these requests, interacts with the SQL database using CRUD operations, and makes additional API calls to external services. The processed data is then returned to the frontend for display and interaction.

f) What does your application use for persistent storage? If a database, provide an ERD or other schema documentation. If file or cloud storage, give details on the structure. Why did you choose this method?

- The application uses an SQL database for persistent storage, with the schema and ERD documented in the project repository. This approach was chosen due to prior experience and its ability to efficiently handle structured data for multiple users. Compared to file-based storage, SQL provides better concurrency control and reduces the risk of data corruption when multiple users interact with the system simultaneously. Security considerations, such as SQL injection, were addressed through careful query handling.

g) How did you implement the requirement for hashing or encryption? What algorithm did you choose and why? Are there any downsides or concerns about the way it is implemented?

- Password security is implemented using PHP's built-in `password_hash` function, which applies a secure hashing algorithm with automatic salting. This approach was chosen because it is a reliable and widely accepted standard without requiring external libraries. One potential concern is that the default algorithm may change between PHP versions, which could lead to inconsistencies over time. However, this is generally mitigated by PHP's support for password verification across algorithm updates.

h) Identify the areas of your application that use an object-oriented component. A class diagram is strongly recommended here.

- Object-oriented components are used throughout the application, particularly in the plant data model and plant search functionality. These classes are reused across multiple features, such as displaying plant data in RecyclerViews and managing plant-specific interactions. This approach improves code organization, reusability, and maintainability.

i) How does user authentication work for your application? What information is stored, what is the process for account creation, login, and management?

- User authentication involves registering with a username, email, and password, which are validated and stored securely in the database. Passwords are hashed before storage, and user sessions are managed with tokens issued upon successful login. These tokens are stored client-side and used to authenticate API requests. Users can register and log in through the same interface, and session management allows continued access without repeated authentication until the session expires.

j) What form does your in-app help and documentation take? Why did you choose this form?

- In-app help is primarily provided through a simple and intuitive user interface, along with Toast messages that communicate errors or important actions in plain language. This approach was chosen to ensure accessibility for users with minimal technical experience, reducing the need for extensive documentation while still providing clear feedback.

k) How did you test your project? Include your formal test plan, results of testing, and explanations for any testing failures.

- The project was tested using a formal test plan that covered key features such as user authentication, plant search, personal collections, watering functionality, weather integration, and security. Testing was done through both manual interaction with the app and reviewing how the code handled API calls, data storage, and UI updates. Most core features performed as expected, including registering/logging in, searching for plants, adding and removing plants, and tracking watering schedules. However, several issues were identified, including missing feedback when search results are empty, inconsistent watering calculations in certain views, and security concerns such as session tokens not expiring and location data being exposed in logs. While the application is largely functional, these failures highlight areas for improvement, particularly in edge cases, user experience, and security, before full deployment.

l) Reflect upon the last 3 months of work, if you had to do it again, what would you do differently? What recommendations do you have for future iterations of this class?

a. Please identify at least three things you liked and want to make sure happen if you had to take the class again

- I liked the responsibility of the class, where after we began, it was in our hands.
- Your regular check-ins were nice, as well as you still holding class in case we wanted to come in to talk about our projects.
- The freedom of choosing whatever languages we liked and what we wanted to make.

b. Please identify at least three things you disliked and wish were not a part of the class.

- I think less of "not part of the class," but I wish there were more communication through announcements or check-ins with possible check-in grades to give us ideas of where we are in the class.
- Change your ways of checking in a bit. I don't know if it was different from others, but when I find problems or issues, it is through talking about what I had done so far with a project. Maybe make students describe what they

have done so far, so that you can point out issues with check-ins and such.

- I think I would have preferred a small presentation in class compared to the video submission we are going to be doing.